

TONE Quarterly Transcript Analysis Code

This code analyzes quarterly earnings call transcripts from companies to measure:

- **Tone/Sentiment:** Is the language positive or negative?
 - **Readability:** How complex is the language?
 - **Uncertainty:** How much uncertain or weak language is used?
-

SECTIONWISE

Step 1-2: Setup and Library Imports

```
!pip install gdown nltk pandas openpyxl  
import os, re, math, pandas as pd  
from google.colab import drive  
import nltk
```

What it does:

- Installs required packages in Google Colab environment
- **gdown:** Downloads files from Google Drive
- **nltk:** Natural Language Toolkit for text processing
- **pandas:** Data manipulation and Excel file creation
- **openpyxl:** Enables writing to Excel files

Interview talking points:

- "We use Google Colab for cloud computing power"
 - "NLTK is the industry standard for text tokenization"
-

Step 3: Mount Google Drive

```
drive.mount('/content/drive', force_remount=True)
```

What it does:

- Connects to Google Drive where transcripts are stored

- `force_remount=True` ensures fresh connection each time

Why this matters:

- Transcripts are large files stored in organized folders
 - Each company has its own folder with quarterly transcripts
-

Step 4-5: Download Loughran-McDonald Dictionary

```
dictionary_url = '1cfg_w3USIRFS97wo7XQmYnuzhpmzboAY'  
gdown.download(id=dictionary_url, output='lm_dictionary.csv')  
dictionary = pd.read_csv('lm_dictionary.csv')
```

What it does:

- Downloads the **Loughran-McDonald Master Dictionary**
- This is a specialized financial sentiment lexicon

Why Loughran-McDonald?

- Designed specifically for financial texts (10-Ks, earnings calls)
- Regular sentiment dictionaries fail with financial language
 - Example: "liability" is negative in finance but neutral elsewhere
 - "outstanding" can be positive or negative in finance
- Academic gold standard for financial text analysis

The dictionary categorizes words into:

1. **Positive:** growth, success, profit, beneficial
2. **Negative:** loss, decline, risk, litigation
3. **Uncertain:** appears, possibly, maybe, uncertain
4. **Weak Modal:** could, might, depending, should

Column name handling:

```
if 'Weak_Modal' in dictionary.columns:  
    modal_weak_col = 'Weak_Modal'  
elif 'WeakModal' in dictionary.columns:  
    modal_weak_col = 'WeakModal'
```

- Different dictionary versions use different column names
- This ensures compatibility across versions

Step 6: Syllable Count Function

```
def syllable_count(word):  
    word = word.lower()  
    count = 0  
    vowels = "aeiouy"  
    # Count vowel groups  
    # Handle special cases (silent 'e', -le endings)
```

What it does:

- Counts syllables in a word for readability calculations
- Used in the **Fog Index** computation

Algorithm:

1. Count vowel groups (consecutive vowels = 1 syllable)
2. Subtract 1 for silent 'e' at end (like "hope" = 1, not 2)
3. Add 1 for "-le" endings preceded by consonant ("table" = 2)
4. Minimum 1 syllable per word

Why syllables matter:

- Complex words have more syllables
 - "Use" (1 syllable) vs "Utilize" (3 syllables)
 - Readability metrics use syllable counts
-

Step 7: Extract Year and Quarter

```
def extract_year_quarter(filename):  
    # Pattern 1: 2023_Q1  
    year_quarter_match = re.search(r'(\d{4})_Q([1-4])', filename)  
    # Pattern 2: Q1_2023  
    quarter_year_match = re.search(r'Q([1-4])_(\d{4})', filename)
```

What it does:

- Parses filenames to extract time period information
- Handles multiple filename formats

Examples:

- `RELIANCE_2023_Q1.txt` → Year: 2023, Quarter: Q1
- `TCS_Q4_2022.txt` → Year: 2022, Quarter: Q4

Why this matters:

- Enables quarterly and yearly trend analysis
 - Can compare Q4 (often most positive) vs Q1
 - Track sentiment changes over time
-

Step 8: Main Analysis Function

:

8a. Text Loading and Cleaning

```
with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
    text = f.read()
clean_text = re.sub(r'^a-zA-Z\s', "", text).lower()
```

What it does:

1. Reads transcript file
2. Removes numbers, punctuation, special characters
3. Converts to lowercase

Why clean?

- "Profit" and "profit!" should be counted as same word
- Numbers don't contribute to sentiment

8b. Tokenization

```
sentences = sent_tokenize(text)
words = word_tokenize(clean_text)
```

What it does:

- **Sentence tokenization:** Splits text into sentences
- **Word tokenization:** Splits text into individual words

Example:

Input: "We saw growth. Revenue increased 20%."

Sentences: ["We saw growth.", "Revenue increased 20%."]

Words: ["we", "saw", "growth", "revenue", "increased"]

8c. Word Counting

positive_count = sum(1 for word in words if word in positive_words)

negative_count = sum(1 for word in words if word in negative_words)

uncertain_count = sum(1 for word in words if word in uncertain_words)

weak_modal_count = sum(1 for word in words if word in weak_modal_words)

What it does:

- Counts how many words fall into each category
- Uses dictionary sets for fast lookup (O(1) time complexity)

Example:

Text: "We might see growth despite risks"

- Positive: 1 (growth)

- Negative: 1 (risks)

- Uncertain: 0

- Weak Modal: 1 (might)

8d. Tone Score Calculation

tone_score = (positive_count - negative_count) / (positive_count + negative_count + 1e-10)

What it does:

- Calculates net sentiment score ranging from -1 to +1
- 1e-10 prevents division by zero

Interpretation:

- +1: Completely positive language
- 0: Balanced or neutral
- -1: Completely negative language

Example:

10 positive words, 5 negative words

Tone = (10-5)/(10+5) = 5/15 = 0.33 (moderately positive)

8e. Ambiguity Percentages

```
percent_uncertain = (uncertain_count / total_words) * 100
percent_weak_modal = (weak_modal_count / total_words) * 100
```

What it does:

- Measures proportion of uncertain/hedging language
- Higher percentages = more ambiguous communication

Why this matters:

- Management hiding bad news uses more uncertain language
- Research shows correlation with future negative events

8f. Readability: File Size

```
size_bytes = os.path.getsize(file_path)
size_mb = size_bytes / (1024 * 1024)
file_size_readability = -math.log(size_mb + 1)
```

What it does:

- Larger files = more verbose = potentially less readable
- Negative log transformation for interpretability

Rationale:

- Managers obfuscating use more words
- Research: longer 10-Ks associated with lower returns

8g. Readability: Fog Index

```
avg_words_per_sentence = total_words / total_sentences
complex_words = [w for w in words if syllable_count(w) >= 3]
percent_complex = (len(complex_words) / total_words) * 100
fog_index = 0.4 * (avg_words_per_sentence + percent_complex)
```

What it does:

- Calculates **Gunning Fog Index** (standard readability metric)
- Formula: $0.4 \times (\text{avg words/sentence} + \% \text{ complex words})$

Interpretation:

- **12:** High school level
- **16:** College level

- **18+:** Graduate level (very difficult)

Example:

20 words/sentence + 30% complex words

Fog = $0.4 \times (20 + 30) = 0.4 \times 50 = 20$ (very difficult)

Step 9: Main Processing Loop

```
root_dir = '/content/drive/MyDrive/BSE_Transcripts_Text/'
for company_folder in company_folders:
    for filename in os.listdir(company_path):
        if filename.endswith('.txt'):
            year, quarter = extract_year_quarter(filename)
            analysis = analyze_transcript(file_path)
```

What it does:

1. Iterates through all company folders
2. Processes each **.txt** transcript file
3. Extracts year/quarter from filename
4. Runs analysis and stores results

Progress tracking:

```
if file_count % 50 == 0:
    print(sample_df[['Company_ID', 'Year', 'Quarter', 'Tone_Score']])
```

- Shows progress every 50 files
- Helps monitor long-running process

Checkpoints:

```
if file_count % 1000 == 0:
    df.to_excel(checkpoint_path, index=False)
```

- Saves intermediate results every 1,000 files
 - Prevents data loss if process crashes
-

Step 10: Statistical Analysis Functions

generate_statistical_analysis()

```
def generate_statistical_analysis(df):  
    desc_stats = df[numeric_cols].describe()  
    quarterly_stats = df.groupby('Quarter')[numeric_cols].agg(['mean', 'median', 'std'])  
    yearly_stats = df.groupby('Year')[numeric_cols].agg(['mean', 'median', 'std'])
```

What it does:

- **Descriptive stats:** Mean, median, std dev, quartiles
- **Quarterly stats:** Aggregations by Q1, Q2, Q3, Q4
- **Yearly stats:** Trends over years

Company rankings:

```
top_tone = company_avg.nlargest(10, 'Tone_Score')  
bottom_tone = company_avg.nsmallest(10, 'Tone_Score')
```

- Identifies best/worst performers
- Useful for identifying outliers

Correlation analysis:

```
correlation_matrix = df[numeric_cols].corr()
```

- Shows relationships between metrics
- Example: Do uncertain transcripts have lower tone scores?

create_summary_insights()

```
def create_summary_insights(df):  
    avg_tone = df['Tone_Score'].mean()  
    if avg_tone > 0:  
        insights.append("Overall sentiment: Positive")
```

What it does:

- Creates human-readable interpretations
 - Identifies patterns (best/worst quarters)
 - Provides context for numbers
-

Step 11: Excel Export

```
with pd.ExcelWriter(output_path, engine='openpyxl') as writer:  
    df.to_excel(writer, sheet_name='Raw_Data', index=False)  
    stats['descriptive_stats'].to_excel(writer, sheet_name='Descriptive_Stats')
```

What it does: Creates multi-sheet Excel workbook:

1. **Raw_Data:** Every transcript's metrics
 2. **Descriptive_Stats:** Overall statistics
 3. **Quarterly_Analysis:** Q1-Q4 comparisons
 4. **Yearly_Analysis:** Year-over-year trends
 5. **Company_Rankings:** Sorted by metrics
 6. **Correlations:** Relationship matrix
 7. **Top_Bottom_Performers:** Best/worst companies
 8. **Summary_Insights:** Text interpretations
-

KEY CONCEPTS

1. Why Financial Sentiment Analysis Matters

- Stock prices react to earnings calls
- Language reveals management confidence
- Predicts future performance
- Used by hedge funds and researchers

2. Technical Decisions

- **Why Loughran-McDonald?** Finance-specific lexicon
- **Why Fog Index?** Standard academic measure
- **Why Google Colab?** Free computing power for large datasets

3. Data Science Pipeline

1. **Data Collection:** Transcripts from BSE
2. **Preprocessing:** Cleaning, tokenization
3. **Feature Engineering:** Tone, readability metrics
4. **Analysis:** Statistical aggregations
5. **Reporting:** Multi-sheet Excel output

4. Potential Research Questions

- Do negative earnings calls predict stock declines?
- Is Q4 more positive than Q1?
- Do larger companies use more complex language?
- Correlation between uncertainty and future volatility?

5. Limitations & Extensions

Limitations:

- Dictionary approach (misses context)
- No speaker separation (CEO vs CFO)
- No comparison to actual financials

Extensions you could mention:

- "Could add machine learning for better sentiment"
- "Could incorporate Q&A session separately"
- "Could link to stock price movements"
- "Could use transformer models (BERT) for context"

QUICK REFERENCE METRICS

Metric	Range	Interpretation
Tone Score	-1 to +1	-1=Very Negative, 0=Neutral, +1=Very Positive
Fog Index	6-20+	12=High School, 16=College, 18+=Graduate
% Uncertain	0-5%+	Higher = More Ambiguous
% Weak Modal	0-3%+	Higher = More Hedging

COMMON QUESTIONS & ANSWERS

Q: Why not use general sentiment tools like VADER? A: Financial language differs from general text. "Liability" is negative in finance but neutral elsewhere. Loughran-McDonald is validated for financial contexts.

Q: How do you handle computational efficiency? A: Use set lookups ($O(1)$), process in batches with checkpoints, leverage pandas vectorization.

Q: What's the biggest challenge? A: Parsing varied filename formats and handling missing/malformed transcripts with error handling.

Q: How would you validate results? A: Compare high-tone transcripts with stock price reactions, correlate with actual quarterly earnings, expert annotation of sample.

Q: What would you improve? A: Add context-aware models (BERT), separate Q&A from presentation, incorporate numerical metrics, temporal analysis.
